

Course Prerequisite Planner

Final Project Struktur Data dan OOP 2026

Topik 9 - Course Prerequisite Planner

Identitas Kelompok

No	Nama	NRP
1	Catur Setyo Ragil	5027251066
2	Aura Syahzanani A	5027251123
3	Donnavie Aulia	5027251093
4	Nur Rizki Syahbana	5027251095
5	Ahmad Rabbani Fata	5027241046

Ringkasan Project

Course Prerequisite Planner adalah aplikasi Java berbasis CLI untuk membantu menyusun rekomendasi urutan pengambilan mata kuliah berdasarkan relasi prasyarat. Mata kuliah direpresentasikan sebagai node pada directed graph, sedangkan relasi prasyarat direpresentasikan sebagai edge berarah.

Program menggunakan dua struktur data utama:

Struktur Data	Implementasi	Fungsi
Directed Graph	<code>Map<String, List<String>></code> adjacency list dan reverse adjacency list	Menyimpan relasi prasyarat, menampilkan graph, DFS prasyarat, cycle detection, dan topological sort
Trie	<code>Trie</code> dan <code>TrieNode</code> buatan sendiri	Search mata kuliah berdasarkan prefix kode atau nama, menghitung jumlah hasil prefix, dan menghapus indeks saat data dihapus/update

Program tidak menggunakan database, GUI, Maven/Gradle, ataupun library graph eksternal. Semua data utama dibaca dari CSV saat runtime.

Deskripsi Masalah

Dalam kurikulum perkuliahan, banyak mata kuliah memiliki prasyarat. Kesalahan dalam membaca relasi prasyarat dapat membuat mahasiswa mengambil mata kuliah lanjutan sebelum fondasinya terpenuhi. Relasi prasyarat juga bisa bermasalah jika membentuk siklus, misalnya mata kuliah A membutuhkan B tetapi B juga membutuhkan A.

Masalah yang diselesaikan program ini:

1. Menampilkan semua mata kuliah beserta atributnya.
2. Mencari mata kuliah berdasarkan prefix kode atau nama.
3. Menampilkan struktur graph prasyarat.
4. Menampilkan prasyarat langsung dan tidak langsung suatu mata kuliah.
5. Memberikan rekomendasi urutan pengambilan mata kuliah.
6. Mendeteksi siklus prasyarat.
7. Mensimulasikan insert, update, delete, dan penambahan relasi prasyarat.

Dataset

Dataset utama berada di folder `data/`:

```
data/courses.csv  
data/prerequisites.csv
```

Jumlah data yang dibaca program:

Jenis Data	Jumlah
Mata kuliah/node	70
Relasi prasyarat/edge	40

Jumlah tersebut memenuhi ketentuan final project: minimal 25 node dan minimal 40 edge.

Format `courses.csv`

Kolom yang tersedia:

Kolom	Keterangan
<code>course_code</code>	Kode unik mata kuliah
<code>course_name</code>	Nama mata kuliah
<code>semester</code>	Semester rekomendasi
<code>sks</code>	Total SKS
<code>theory_sks</code>	SKS teori
<code>practical_sks</code>	SKS praktikum
<code>category</code>	Kategori mata kuliah
<code>track</code>	Jalur/kelompok mata kuliah
<code>difficulty</code>	Tingkat kesulitan
<code>is_node</code>	Penanda bahwa baris adalah node graph

Contoh:

```
course_code,course_name,semester,sks,theory_sks,practical_sks,category,track,difficulty,is_node
ET234103,Algoritma dan Teknik
Pemrograman,1,4,3,1,Pemrograman,Core,Dasar,TRUE
```

Format `prerequisites.csv`

Kolom utama yang dipakai program:

Kolom	Keterangan
<code>prerequisite_code</code>	Kode mata kuliah prasyarat
<code>course_code</code>	Kode mata kuliah tujuan

Kolom lain seperti `edge_id`, `relation_type`, `is_official`, dan `relation_reason` dipakai sebagai dokumentasi dataset.

Contoh:

```
edge_id,prerequisite_code,prerequisite_name,course_code,course_name,relation_type,is_official,edge_direction,relation_reason
E003,ET234103,Algoritma dan Teknik Pemrograman,ET234203,Struktur Data dan Pemrograman Berorientasi Objek,OFFICIAL,TRUE,prerequisite_code -> course_code,Prasyarat resmi: Struktur Data dan PBO mensyaratkan Algoritma dan Teknik Pemrograman
```

Struktur Folder

```
course-prerequisite-planner/
├── src/
│   ├── Main.java
│   ├── graph/
│   │   └── CourseGraph.java
│   ├── model/
│   │   └── Course.java
│   └── tree/
│       ├── Trie.java
│       └── TrieNode.java
├── data/
│   ├── courses.csv
│   ├── prerequisites.csv
│   └── graph_dataset.csv
├── assets/
│   └── screenshot-*.png
├── docs/
│   ├── laporan.pdf
│   └── tracing.pdf
```

├─

 README.md

└─

 TRACING.md

Cara Compile dan Run

Jalankan dari root folder project:

```
javac -d out src/Main.java src/model/Course.java src/graph/CourseGraph.java
src/tree/Trie.java src/tree/TrieNode.java
java -cp out Main
```

Alternatif untuk Linux/macOS/Git Bash:

```
javac -d out $(find src -name "*.java")
java -cp out Main
```

Jika ingin menggunakan path dataset lain:

```
java -cp out Main data/courses.csv data/prerequisites.csv
```

Untuk Windows PowerShell:

```
javac -d out (Get-ChildItem -Recurse src -Filter *.java | ForEach-Object {
$_.FullName })
java -cp out Main
```

Desain Class

File	Tanggung Jawab
src/model/Course.java	Model immutable untuk menyimpan data mata kuliah. Constructor memvalidasi kode, nama, semester, SKS, kategori, track, dan difficulty.
src/graph/CourseGraph.java	Membaca CSV, menyimpan directed graph, menambah/menghapus/update course, menambah edge, DFS prasyarat, cycle detection, dan topological sort.
src/tree/Trie.java	Insert, search prefix, count match, dan delete indeks course berdasarkan kode/nama.
src/tree/TrieNode.java	Node Trie yang menyimpan child map dan daftar kode mata kuliah pada prefix tertentu.

File	Tanggung Jawab
src/Main.java	Menu CLI, input user, format tabel, integrasi Trie dan Graph.

Representasi Graph

Graph yang digunakan adalah directed graph dengan arah edge:

```
prerequisite_code -> course_code
```

Artinya:

```
A -> B
A adalah prasyarat untuk B
```

Implementasi di **CourseGraph**:

```
Map<String, Course> courses;
Map<String, List<String>> adjacencyList;
Map<String, List<String>> reverseAdjacencyList;
```

Fungsi masing-masing struktur:

Struktur	Fungsi
courses	Menyimpan detail mata kuliah berdasarkan kode
adjacencyList	Menyimpan relasi dari prasyarat ke mata kuliah lanjutan
reverseAdjacencyList	Menyimpan relasi dari mata kuliah ke daftar prasyarat langsung

Alasan memakai adjacency list:

- 1. Relasi prasyarat cenderung sparse, sehingga adjacency list lebih hemat memori dibanding adjacency matrix.
- 2. Traversal DFS dan cycle detection berjalan efisien dalam $O(V + E)$, sedangkan topological sort memakai prioritas semester agar urutan rekomendasi lebih natural.
- 3. Data mudah ditampilkan dalam bentuk **prasyarat -> daftar mata kuliah lanjutan**.

Struktur Tree

Tree yang digunakan adalah Trie. Setiap karakter dari kode dan nama mata kuliah disimpan sebagai jalur node. Setiap node menyimpan kumpulan kode mata kuliah yang cocok dengan prefix tersebut.

Contoh penggunaan:

```
Input prefix: ET2343
Output:
ET234301
ET234302
ET234303
ET234304
ET234305
ET234306
```

Operasi Trie yang diimplementasikan:

Method	Fungsi	Kompleksitas
<code>insert(String key, String courseCode)</code>	Mengindeks kode atau nama mata kuliah	O(L)
<code>searchByPrefix(String prefix)</code>	Mengambil semua kode mata kuliah pada prefix	O(L + R)
<code>countMatches(String prefix)</code>	Menghitung jumlah hasil prefix	O(L + R)
<code>delete(String key, String courseCode)</code>	Menghapus indeks saat course dihapus/update	O(L)

Keterangan:

- L = panjang kode/nama/prefix.
- R = jumlah hasil yang dikembalikan.

Algoritma Graph

1. DFS untuk Semua Prasyarat

DFS digunakan untuk mencari seluruh prasyarat langsung dan tidak langsung. Traversal dilakukan pada `reverseAdjacencyList` karena struktur tersebut menyimpan relasi dari mata kuliah ke prasyaratnya.

Contoh:

```
ET234103 -> ET234203 -> ET234305
```

Jika dicari semua prasyarat `ET234305`, maka DFS pada reverse graph menemukan:

```
ET234203
ET234103
```

2. Cycle Detection

Cycle detection menggunakan DFS dengan status kunjungan:

Status	Arti
0	Belum dikunjungi
1	Sedang dikunjungi
2	Selesai dikunjungi

Jika DFS menemukan edge menuju node berstatus **1**, maka graph memiliki cycle. Program menolak relasi baru yang menyebabkan cycle dan melakukan rollback edge.

3. Topological Sort

Topological sort menggunakan Kahn's Algorithm. Implementasi memakai **PriorityQueue** berdasarkan semester rekomendasi dan kode mata kuliah agar course semester rendah diprioritaskan ketika beberapa node sama-sama sudah bebas prasyarat.

1. Hitung indegree setiap mata kuliah.
2. Masukkan semua mata kuliah dengan indegree 0 ke priority queue.
3. Ambil node prioritas tertinggi dari queue dan masukkan ke hasil urutan.
4. Kurangi indegree semua tetangga keluar.
5. Jika indegree tetangga menjadi 0, masukkan ke queue.
6. Ulangi sampai queue kosong.

Program selalu menjalankan cycle detection sebelum topological sort. Jika ada cycle, rekomendasi urutan tidak dibuat.

Fitur Program

Menu CLI:

- ```
1. Tampilkan semua mata kuliah
2. Search prefix kode/nama
3. Tampilkan adjacency list graph
4. Tampilkan prasyarat langsung
5. Tampilkan semua prasyarat DFS
6. Rekomendasi topological sort
7. Deteksi siklus prasyarat
8. Tambah mata kuliah
9. Tambah relasi prasyarat
10. Update mata kuliah
11. Delete mata kuliah
0. Keluar
```

Fitur yang menggabungkan Tree dan Graph terdapat pada menu search prefix. Trie dipakai untuk menemukan kode mata kuliah berdasarkan prefix, lalu Graph dipakai untuk mengambil detail mata kuliah, jumlah prasyarat langsung, dan jumlah mata kuliah lanjutan.

## Design Decision Log

| No | Keputusan                                  | Alternatif                                     | Alasan Memilih                                                                      | Risiko/Kelemahan                                     |
|----|--------------------------------------------|------------------------------------------------|-------------------------------------------------------------------------------------|------------------------------------------------------|
| 1  | Directed graph                             | Undirected graph                               | Relasi prasyarat punya arah yang jelas dari prasyarat ke mata kuliah tujuan         | Salah arah edge membuat topological sort tidak valid |
| 2  | Adjacency list                             | Adjacency matrix                               | Lebih hemat memori untuk graph sparse                                               | Cek edge langsung perlu scan list tetangga           |
| 3  | Reverse adjacency list                     | Traversal mundur dari adjacency normal         | Prasyarat langsung dan DFS prasyarat jadi lebih sederhana                           | Memori bertambah $O(E)$                              |
| 4  | Trie untuk search prefix                   | HashMap biasa                                  | HashMap tidak efisien untuk prefix search                                           | Trie memakai memori lebih besar                      |
| 5  | Kahn's Algorithm dengan prioritas semester | DFS-based topological sort                     | Mudah dijelaskan dengan indegree dan hasil lebih natural untuk rekomendasi semester | Operasi queue menjadi $O(\log V)$                    |
| 6  | Reject edge yang menyebabkan cycle         | Membiarkan cycle lalu hanya memberi peringatan | Menjaga data prasyarat tetap valid                                                  | Add edge menjadi lebih mahal karena ada cycle check  |
| 7  | CSV runtime                                | Hardcode Java                                  | Dataset bisa diubah tanpa mengubah source code                                      | Format CSV harus divalidasi                          |
| 8  | Kode course tidak boleh diupdate           | Mengubah semua key graph                       | Kode adalah identitas node dan key relasi                                           | Jika kode salah, perlu delete lalu insert ulang      |

## Edge Case yang Ditangani

| Edge Case                 | Penanganan Program                                                           |
|---------------------------|------------------------------------------------------------------------------|
| File CSV tidak ditemukan  | Ditangkap sebagai <code>IOException</code> , program menampilkan pesan error |
| Header CSV tidak sesuai   | Program menolak load dan menyebut kolom wajib yang hilang                    |
| Semester/SKS tidak valid  | Program menolak angka kosong, non-numerik, nol, atau negatif                 |
| Kode mata kuliah duplikat | Program menolak data course duplikat                                         |
| Relasi prasyarat duplikat | Program tidak menambahkan edge yang sama dua kali                            |



| Edge Case                                | Penanganan Program                                              |
|------------------------------------------|-----------------------------------------------------------------|
| Relasi ke course tidak ada               | Program menolak edge                                            |
| Course menjadi prasyarat dirinya sendiri | Program menolak edge                                            |
| Relasi baru menyebabkan cycle            | Program rollback edge dan menampilkan error                     |
| Prefix tidak ditemukan                   | Program menampilkan warning dan kembali ke menu                 |
| Delete course yang punya banyak edge     | Program menghapus edge masuk dan keluar terkait course tersebut |
| Topological sort saat ada cycle          | Program membatalkan rekomendasi                                 |

## Analisis Kompleksitas

Notasi:

V = jumlah mata kuliah/node

E = jumlah relasi prasyarat/edge

L = panjang kode atau nama mata kuliah

R = jumlah hasil prefix

k = jumlah tetangga pada list adjacency

| Operasi                             | Struktur/Algoritma               | Kompleksitas          | Alasan                                                                                    |
|-------------------------------------|----------------------------------|-----------------------|-------------------------------------------------------------------------------------------|
| Load <code>courses.csv</code>       | Map                              | $O(V)$                | Membaca setiap baris course satu kali                                                     |
| Load <code>prerequisites.csv</code> | Graph + cycle check              | $O(E \times (V + E))$ | Setiap edge ditambahkan lalu dicek cycle agar data tetap valid                            |
| Insert course                       | Map + adjacency list             | $O(1)$ rata-rata      | Insert ke <code>LinkedHashMap</code> dan inialisasi list                                  |
| Add edge                            | Adjacency list + DFS cycle       | $O(k + V + E)$        | Cek duplikat pada list tetangga, lalu cycle detection                                     |
| Direct prerequisite                 | Reverse adjacency list           | $O(k)$                | Mengambil list prasyarat langsung                                                         |
| All prerequisite                    | DFS reverse graph                | $O(V + E)$            | Menelusuri node dan edge yang relevan                                                     |
| Cycle detection                     | DFS status 0/1/2                 | $O(V + E)$            | Setiap node dan edge dikunjungi maksimal sekali                                           |
| Topological sort                    | Kahn's Algorithm + PriorityQueue | $O((V + E) \log V)$   | Setiap node masuk/keluar priority queue, dan setiap edge diproses saat indegree dikurangi |

| Operasi            | Struktur/Algoritma | Kompleksitas | Alasan                                                        |
|--------------------|--------------------|--------------|---------------------------------------------------------------|
| Insert Trie        | Trie               | $O(L)$       | Menelusuri karakter key                                       |
| Search prefix Trie | Trie               | $O(L + R)$   | Menelusuri prefix dan menyalin hasil                          |
| Delete dari Trie   | Trie               | $O(L)$       | Menghapus course code di jalur prefix dan pruning node kosong |
| Delete course      | Graph              | $O(V + E)$   | Menghapus node dan semua edge masuk/keluar terkait            |

## Skenario Pengujian

### Skenario Normal

| No | Skenario                              | Expected Output                                  |
|----|---------------------------------------|--------------------------------------------------|
| 1  | Jalankan program                      | Muncul jumlah mata kuliah 70 dan relasi 40       |
| 2  | Search prefix ET2343                  | Muncul 6 mata kuliah semester 3                  |
| 3  | Tampilkan prasyarat langsung ET234203 | Muncul ET234103 sebagai prasyarat                |
| 4  | Tampilkan semua prasyarat ET234801    | Muncul rantai prasyarat menuju Tugas Akhir       |
| 5  | Jalankan topological sort             | Muncul urutan mata kuliah dari dasar ke lanjutan |
| 6  | Jalankan cycle detection              | Muncul status graph tidak memiliki siklus        |

### Skenario Edge Case

| No | Skenario                                      | Expected Output                            |
|----|-----------------------------------------------|--------------------------------------------|
| 1  | Tambah relasi ET234801 -> ET234103            | Ditolak karena menyebabkan cycle           |
| 2  | Search prefix yang tidak ada                  | Muncul warning tidak ada mata kuliah cocok |
| 3  | Tambah mata kuliah dengan kode yang sudah ada | Ditolak karena kode sudah ada              |
| 4  | Tambah relasi ke kode yang tidak ada          | Ditolak karena course tidak ditemukan      |
| 5  | Delete mata kuliah yang punya edge            | Course dan semua edge terkait terhapus     |

## Screenshot Hasil Program

### Menu Utama

```

course-prerequisite-planner > java -cp out Main
=====
 COURSE PREREQUISITE PLANNER
 Topik 9 - Struktur Data dan OOP
=====
Dataset mata kuliah : data/courses.csv
Dataset prasyarat : data/prerequisites.csv
Jumlah mata kuliah : 70
Jumlah relasi : 40
Status graph : Tidak ada siklus
=====

 MENU UTAMA
=====
Mata kuliah: 70 | Relasi prasyarat: 40 | Cycle: TIDAK

1. Tampilkan semua mata kuliah 7. Deteksi siklus prasyarat
2. Search prefix kode/nama 8. Tambah mata kuliah
3. Tampilkan adjacency list graph 9. Tambah relasi prasyarat
4. Tampilkan prasyarat langsung 10. Update mata kuliah
5. Tampilkan semua prasyarat DFS 11. Delete mata kuliah
6. Rekomendasi topological sort 0. Keluar

Pilih menu:

```

## Search Prefix

```

Pilih menu: 2
=====
Search Prefix dengan Trie
=====
Masukkan prefix kode/nama: ET2343
Prefix : ET2343
Jumlah hasil : 6
Sumber data : Trie untuk pencarian, Graph untuk detail dan relasi

```

| Kode     | Nama Mata Kuliah                       | Sem | SKS | Kategori             | Track | Difficulty |
|----------|----------------------------------------|-----|-----|----------------------|-------|------------|
| ET234301 | Ethical Hacking dan Uji Keamanan Siber | 3   | 4   | Keamanan Siber       | Core  | Lanjut     |
| ET234302 | Keamanan Jaringan Komputer             | 3   | 3   | Keamanan Siber       | Core  | Lanjut     |
| ET234303 | Internet of Things                     | 3   | 3   | IoT dan Smart City   | Core  | Lanjut     |
| ET234304 | Komunikasi Profesional                 | 3   | 2   | Komunikasi           | Core  | Dasar      |
| ET234305 | Pemrograman Web                        | 3   | 4   | Pengembangan Apli... | Core  | Lanjut     |
| ET234306 | Komunikasi Data dan Jaringan Komputer  | 3   | 4   | Jaringan             | Core  | Lanjut     |

```

Ringkasan relasi hasil search:
- ET234301 | prasyarat langsung: 1 | mata kuliah lanjutan: 1
- ET234302 | prasyarat langsung: 0 | mata kuliah lanjutan: 3
- ET234303 | prasyarat langsung: 2 | mata kuliah lanjutan: 2
- ET234304 | prasyarat langsung: 0 | mata kuliah lanjutan: 0
- ET234305 | prasyarat langsung: 2 | mata kuliah lanjutan: 4
- ET234306 | prasyarat langsung: 1 | mata kuliah lanjutan: 2

```

## Prasyarat Langsung

```

Pilih menu: 4
=====
Prasyarat Langsung
=====
Masukkan kode mata kuliah: ET234203
Prasyarat langsung untuk ET234203 - Struktur Data dan Pemrograman Berorientasi Objek | Sem 2 | 3 SKS | Pemrograman | Core | Menengah
- ET234103 - Algoritma dan Teknik Pemrograman | Sem 1 | 4 SKS | Pemrograman | Core | Dasar

```

## Semua Prasyarat dengan DFS

```

Pilih menu: 5
=====
Semua Prasyarat dengan DFS
=====
Masukkan kode mata kuliah: ET234305
Target: ET234305 - Pemrograman Web | Sem 3 | 4 SKS | Pengembangan Aplikasi | Core | Lanjut

Kode | Nama Mata Kuliah | Sem | SKS | Kategori | Track | Difficulty

ET234203 | Struktur Data dan Pemrograman Berorientasi O... | 2 | 3 | Pemrograman | Core | Menengah
ET234103 | Algoritma dan Teknik Pemrograman | 1 | 4 | Pemrograman | Core | Dasar

```

## Topological Sort

```

Pilih menu: 6
=====
Rekomendasi Urutan Pengambilan Mata Kuliah
=====
No | Kode | Nama Mata Kuliah | Sem | SKS

0001 | EE234101 | Pengantar Teknologi Elektro dan Informatika ... | 1 | 2
0002 | ET234101 | Statistika dan Probabilitas | 1 | 2
0003 | ET234102 | Arsitektur dan Organisasi Komputer | 1 | 3
0004 | ET234103 | Algoritma dan Teknik Pemrograman | 1 | 4
0005 | ET234104 | Hukum dan Etika Teknologi Informasi | 1 | 2
0006 | SF234204 | Fisika Dasar | 1 | 3
0007 | SM234101 | Kalkulus 1 | 1 | 3
0008 | ET234201 | Sistem Operasi | 2 | 4
0009 | ET234202 | Arsitektur Enterprise | 2 | 3
0010 | ET234203 | Struktur Data dan Pemrograman Berorientasi O... | 2 | 3
0011 | ET234204 | Sistem Basis Data | 2 | 4
0012 | SM234201 | Kalkulus 2 | 2 | 3
0013 | ET234301 | Ethical Hacking dan Uji Keamanan Siber | 3 | 4
0014 | ET234302 | Keamanan Jaringan Komputer | 3 | 3
0015 | ET234303 | Internet of Things | 3 | 3
0016 | ET234304 | Komunikasi Profesional | 3 | 2
0017 | ET234305 | Pemrograman Web | 3 | 4
0018 | ET234306 | Komunikasi Data dan Jaringan Komputer | 3 | 4
0019 | ET234401 | Manajemen Insiden Keamanan Siber | 4 | 3
0020 | ET234402 | Security Operations Center | 4 | 3
0021 | ET234403 | Teknologi Komputasi Awan | 4 | 4
0022 | ET234404 | Big Data dan Data Lakehouse | 4 | 3
0023 | ET234405 | Kecerdasan Artifisial dan Machine Learning | 4 | 4
0024 | ET234406 | Integrasi Sistem | 4 | 3
0025 | ET234501 | Kriptografi | 5 | 3
0026 | ET234502 | Keamanan Web dan Aplikasi | 5 | 3
0027 | ET234503 | Tata Kelola Teknologi Informasi | 5 | 3
0028 | ET234504 | Teknologi Smartcity | 5 | 4
0029 | ET234505 | Strategi Optimasi Komputasi Awan | 5 | 3
0030 | ET234506 | Pemrograman Mobile | 5 | 3
0031 | ET234601 | Interaksi Manusia dan Komputer | 6 | 3
0032 | ET234602 | Pengembangan Sistem/Aplikasi (Capstone Project) | 6 | 3
0033 | ET234611 | Kuliah Kerja Nyata Tematik | 6 | 3
0034 | ET234612 | Teknologi Multimedia | 6 | 3
0035 | ET234614 | Operasional Pengembang | 6 | 3
0036 | ET234615 | Desain dan Manajemen Jaringan Komputer | 6 | 3
0037 | ET234616 | Jaringan Nirkabel | 6 | 3

```



|      |          |                                                 |   |   |
|------|----------|-------------------------------------------------|---|---|
| 0038 | UG234901 | Agama Islam                                     | 6 | 2 |
| 0039 | UG234902 | Agama Kristen                                   | 6 | 2 |
| 0040 | UG234903 | Agama Katolik                                   | 6 | 2 |
| 0041 | UG234904 | Agama Hindu                                     | 6 | 2 |
| 0042 | UG234905 | Agama Budha                                     | 6 | 2 |
| 0043 | UG234906 | Agama Konghucu                                  | 6 | 2 |
| 0044 | UG234913 | Kewarganegaraan                                 | 6 | 2 |
| 0045 | UG234916 | Aplikasi Teknologi dan Transformasi Digital     | 6 | 3 |
| 0046 | ET234701 | Kerja Praktik                                   | 7 | 2 |
| 0047 | ET234702 | Pra-TA / Metodologi Penulisan Ilmiah            | 7 | 2 |
| 0048 | ET234703 | Manajemen Proyek                                | 7 | 3 |
| 0049 | ET234711 | Business Intelligence                           | 7 | 3 |
| 0050 | ET234712 | Forensik Digital                                | 7 | 3 |
| 0051 | ET234713 | Malware dan Cyber Threat Intelligence           | 7 | 3 |
| 0052 | ET234714 | Manajemen Keamanan Siber                        | 7 | 3 |
| 0053 | ET234715 | Quantum Computing untuk Cybersecurity           | 7 | 3 |
| 0054 | ET234716 | Privasi/Perlindungan Data dan Ergonomi pada ... | 7 | 3 |
| 0055 | ET234717 | Audit Keamanan Informasi                        | 7 | 3 |
| 0056 | ET234718 | Skalabilitas dan Reliabilitas Sistem            | 7 | 3 |
| 0057 | UG234911 | Pancasila                                       | 7 | 2 |
| 0058 | UG234912 | Bahasa Indonesia                                | 7 | 2 |
| 0059 | UG234914 | Bahasa Inggris                                  | 7 | 2 |
| 0060 | UG234915 | Kewirausahaan Berbasis Teknologi                | 7 | 2 |
| 0061 | ET234801 | Tugas Akhir                                     | 8 | 6 |
| 0062 | ET234811 | Pengadaan IT                                    | 8 | 3 |
| 0063 | ET234812 | Penjaminan Kualitas TI                          | 8 | 3 |
| 0064 | ET234813 | Teknologi Blockchain                            | 8 | 3 |
| 0065 | ET234814 | Sistem Cerdas                                   | 8 | 3 |
| 0066 | ET234815 | Otomasi Sistem                                  | 8 | 3 |
| 0067 | ET234816 | Keamanan Sistem Cyber-Physical dan Kota Cerdas  | 8 | 3 |
| 0068 | ET234821 | Magang Bidang Keamanan Siber                    | 8 | 2 |
| 0069 | ET234822 | Magang Bidang Pengembangan Aplikasi             | 8 | 2 |
| 0070 | ET234823 | Magang Bidang Kreasi Digital                    | 8 | 2 |

## Cycle Detection

```
Pilih menu: 7
=====
Cycle Detection
=====
[OK] Graph tidak memiliki siklus. Topological sort aman dijalankan.
```

## Relasi Ditolak karena Cycle

```
Pilih menu: 9
=====
Tambah Relasi Prasyarat
=====
Kode prasyarat: ET234305
Kode mata kuliah tujuan: ET234103
[ERROR] Gagal menambah relasi: Relasi menyebabkan cycle dan dibatalkan: ET234305 -> ET234103
```

## What-if Analysis

1. Apa yang terjadi jika jumlah node naik menjadi 10.000?

DFS dan cycle detection tetap berskala linear  $O(V + E)$ . Topological sort memakai priority queue sehingga kompleksitasnya  $O((V + E) \log V)$ , tetapi hasilnya lebih natural karena memprioritaskan semester rekomendasi lebih kecil. Bottleneck utama adalah proses `addEdge` saat load karena program mengecek cycle setiap edge. Untuk dataset sangat besar, optimasi yang lebih baik adalah membaca semua edge terlebih dahulu, lalu menjalankan satu cycle detection global.

## 2. Apa yang terjadi jika edge tertentu dihapus?

Indegree mata kuliah tujuan akan berkurang. Hasil topological sort dapat berubah karena mata kuliah yang sebelumnya bergantung pada prasyarat tertentu bisa muncul lebih awal. Jika edge yang dihapus adalah bagian dari cycle, maka cycle dapat hilang.

## 3. Apa yang terjadi jika bobot edge ditambahkan?

Topological sort tidak membutuhkan bobot, sehingga rekomendasi prasyarat tetap bisa berjalan. Namun jika bobot dipakai untuk menghitung tingkat kesulitan transisi atau prioritas pengambilan, struktur edge perlu diubah dari `List<String>` menjadi list object `Edge` yang menyimpan tujuan dan bobot.

## 4. Apa yang terjadi jika input user tidak ditemukan?

Program menampilkan pesan warning dan kembali ke menu. Operasi graph tidak dijalankan pada kode yang tidak ada, sehingga program tidak crash.

## 5. Apa yang terjadi jika data duplikat dimasukkan?

Course dengan kode duplikat ditolak karena kode adalah key graph. Relasi prasyarat duplikat tidak ditambahkan ulang. Penanganan ini menjaga hasil DFS dan topological sort agar tidak terdistorsi oleh edge ganda.

## 6. Apa yang terjadi jika graph tidak terhubung?

Program tetap berjalan. Cycle detection mengecek semua komponen, sedangkan topological sort memasukkan semua node dengan indegree 0 ke queue. Mata kuliah yang tidak punya relasi prasyarat akan tetap muncul dalam rekomendasi.